

GATE Data Science & Artificial Intelligence (DA)

Question Bank: Elementary Sorting Algorithms & Minimax Trees

10 Selected PYQs (GATE DA / CS) + 10 Targeted GATE-Level Practice Questions

Topic Coverage:

1. **Topic 1:** Sorting I: Bubble, Insertion, and Selection Sort (Comparisons, Swaps, Best/Worst Cases, In-place & Stability)
2. **Topic 2:** Sorting Complexity Table Memorization & 3 Array Trace Problems
3. **Topic 3:** Adversarial Search: Two-Player Zero-Sum Games, Minimax Principle, and Evaluation Rules
4. **Topic 4:** Tracing Depth-2 Minimax Trees and Optimal Value Backpropagation

Part I: 10 Previous Year Questions (PYQs)

Q1. Insertion Sort on Nearly Sorted Input [GATE CS 2021 • MCQ • 1 Mark] Consider an array of n elements where each element is at most k positions away from its target position in the sorted array ($k \ll n$). What is the running time of Insertion Sort on this array?

- (A) $\Theta(n)$
- (B) $\Theta(n \cdot k)$
- (C) $\Theta(n \log k)$
- (D) $\Theta(n^2)$

Answer: (B)

Solution / Explanation:

In Insertion Sort, each element is shifted leftward until it reaches its correct relative position among the previously sorted elements. Since every element is at most k positions away from its final sorted location, each of the n elements undergoes at most k comparisons and shifts. Therefore, the total number of operations is bounded by $\sum_{i=1}^n \mathcal{O}(k) = \Theta(n \cdot k)$.

Q2. Selection Sort Swap Count Invariant

[GATE CS 2019 • MCQ • 1 Mark] Which of the

following elementary sorting algorithms performs at most $\mathcal{O}(n)$ swaps (data item movements) in the worst case on an array of n elements?

- (A) Selection Sort
- (B) Insertion Sort
- (C) Bubble Sort
- (D) Neither

Answer: (A)

Solution / Explanation:

Selection Sort iterates $n - 1$ times. In each pass i , it identifies the minimum element in the unsorted suffix $A[i \dots n - 1]$ using comparisons and performs exactly one swap to place it at index i . Thus, the total number of swaps is at most $n - 1 = \Theta(n)$ in all cases (best, average, and worst). In contrast, Bubble Sort and Insertion Sort can perform up to $\Theta(n^2)$ swaps/shifts.

Q3. Sorting Trace: Insertion Sort Iteration State

[GATE CS 2016 • MCQ •

2 Marks] An array $A = [7, 4, 8, 2, 5]$ is sorted using standard Insertion Sort. Which of the following represents the exact state of the array after the completion of the 2nd outer loop pass (inserting the element originally at index 2)?

- (A) $[4, 7, 8, 2, 5]$
- (B) $[2, 4, 7, 8, 5]$
- (C) $[4, 7, 2, 8, 5]$
- (D) $[7, 4, 8, 2, 5]$

Answer: (A)

Solution / Explanation:

Let 0-based indexing be used:

- Initial array: $[7, 4, 8, 2, 5]$
- **Pass 1 ($i = 1$, key = 4):** Compare 4 with 7. Shift 7 right, insert 4 at index 0.
Array becomes: $[4, 7, 8, 2, 5]$.
- **Pass 2 ($i = 2$, key = 8):** Compare 8 with 7. Since $8 > 7$, no shifts occur. Key 8 remains at index 2.
Array becomes: $[4, 7, 8, 2, 5]$.

After the 2nd pass, the array state is $[4, 7, 8, 2, 5]$.

Q4. Minimax Tree: Root Value Evaluation at Depth 2

[GATE DA 2024 /

CS AI Scope • NAT • 2 Marks] Consider a 2-ply (depth 2) game tree where the root is a MAX node with two children B and C (both MIN nodes). Node B has terminal children with payoffs 3 and 8. Node C has terminal children with payoffs 6 and 2. What is the backed-up minimax value at the root node?

Answer: 3

Solution / Explanation:

Let us evaluate layer by layer from the terminal leaves:

- Node B is a MIN node with children $\{3, 8\}$:

$$\text{Value}(B) = \min(3, 8) = 3.$$

- Node C is a MIN node with children $\{6, 2\}$:

$$\text{Value}(C) = \min(6, 2) = 2.$$

- Root A is a MAX node choosing between children B and C :

$$\text{Value}(A) = \max(\text{Value}(B), \text{Value}(C)) = \max(3, 2) = 3.$$

The backed-up minimax value at the root is 3.

Q5. Bubble Sort Best-Case with Early Termination Flag

[GATE CS 2017 • MCQ •

1 Mark] A version of Bubble Sort maintains a boolean flag `swapped`. If no swaps occur during an entire inner loop pass, the algorithm terminates immediately. What is the time complexity of this optimized Bubble Sort on an array that is already sorted in non-decreasing order?

- (A) $\Theta(1)$
- (B) $\Theta(n)$
- (C) $\Theta(n \log n)$
- (D) $\Theta(n^2)$

Answer: (B)

Solution / Explanation:

On an already sorted array of size n , the first pass compares each adjacent pair: $(A[0], A[1]), (A[1], A[2]), \dots, (A[n-2], A[n-1])$, making exactly $n-1$ comparisons. Because no adjacent pair is inverted, no swap occurs and `swapped` remains `False`. The algorithm detects this and halts after the single pass. Total time is $\Theta(n)$.

Q6. Sorting Trace: Bubble Sort Swap Count NAT [GATE CS 2015 • NAT • 2 Marks] Consider the array $A = [5, 1, 4, 2, 8]$. How many total adjacent swaps are performed by standard Bubble Sort to completely sort this array into ascending order?

Answer: 4

Solution / Explanation:

Let us trace each pass of Bubble Sort:

• **Pass 1:**

- Compare (5, 1): $5 > 1 \implies$ **Swap 1.** Array: [1, 5, 4, 2, 8]
- Compare (5, 4): $5 > 4 \implies$ **Swap 2.** Array: [1, 4, 5, 2, 8]
- Compare (5, 2): $5 > 2 \implies$ **Swap 3.** Array: [1, 4, 2, 5, 8]
- Compare (5, 8): $5 < 8 \implies$ No swap.

• **Pass 2:**

- Compare (1, 4): $1 < 4 \implies$ No swap.
- Compare (4, 2): $4 > 2 \implies$ **Swap 4.** Array: [1, 2, 4, 5, 8]
- Compare (4, 5): $4 < 5 \implies$ No swap.

- **Pass 3:** Array is already [1, 2, 4, 5, 8]. No adjacent elements inverted $\implies$ 0 swaps.

Total number of swaps = $3 + 1 = 4$. (Note: The number of swaps in Bubble Sort equals the number of inversions in the original array: pairs (5, 1), (5, 4), (5, 2), (4, 2) $\implies$ 4 inversions).

Q7. Number of Leaf Nodes in Complete Minimax Tree

[GATE CS 2018 / AI Scope

• **NAT • 1 Mark]** A two-player zero-sum game has a uniform legal move branching factor $b = 3$. If the game tree is expanded completely to depth $d = 2$ (where the root is at depth 0), what is the total number of terminal leaf evaluation nodes?

Answer: 9

Solution / Explanation:

In a uniform tree with branching factor b and depth d , each node at depth k has b children. The number of leaf nodes at depth d is given by:

$$N_{\text{leaves}} = b^d = 3^2 = 9.$$

Q8. Selection Sort Stability and In-Place Properties

[GATE CS 2020

• **MSQ • 2 Marks]** Which of the following statements is/are **TRUE** regarding the standard in-place implementation of Selection Sort?

- (A) The number of comparisons performed is $\Theta(n^2)$ even if the input array is already sorted.
- (B) It requires $\mathcal{O}(1)$ auxiliary space.
- (C) The standard array-based swap implementation of Selection Sort is stable.
- (D) The best-case and worst-case time complexities of Selection Sort are identical asymptotically.

Answer: (A, B, D)

Solution / Explanation:

- (A) is True: Selection Sort always scans the remaining unsorted suffix regardless of ordering: $\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2} = \Theta(n^2)$ comparisons.
- (B) is True: It operates in-place using only index pointers, requiring $\mathcal{O}(1)$ auxiliary memory.
- (C) is False: Standard Selection Sort is **unstable**. For example, in $[2_a, 2_b, 1]$, swapping 2_a with 1 yields $[1, 2_b, 2_a]$, inverting the relative order of identical keys.
- (D) is True: Both best-case and worst-case time complexities are $\Theta(n^2)$.

Q9. Zero-Sum Game Property in Minimax Search [GATE CS 2014 / AI Scope • MCQ • 1 Mark] In a two-player zero-sum game with players MAX and MIN, if the utility payoff to MAX at a terminal state is U , what is the utility payoff to player MIN at the same terminal state?

- (A) U
- (B) $-U$
- (C) $\frac{1}{U}$
- (D) 0

Answer: (B)

Solution / Explanation:

In a zero-sum game, the total payoff distributed among all players sums to zero at all terminal outcomes:

$$U_{\text{MAX}} + U_{\text{MIN}} = 0 \implies U_{\text{MIN}} = -U_{\text{MAX}} = -U.$$

Any gain by player MAX represents an equivalent loss to player MIN.

Q10. Sorting Trace: Selection Sort 2nd Pass State [GATE CS 2013 • MCQ • 1 Mark] Consider the array $A = [64, 25, 12, 22, 11]$. Selection Sort is applied to sort the array in ascending order. What is the state of the array after the completion of the 2nd pass?

- (A) $[11, 12, 25, 22, 64]$
- (B) $[11, 12, 64, 22, 25]$
- (C) $[11, 25, 12, 22, 64]$
- (D) $[12, 11, 22, 25, 64]$

Answer: (A)

Solution / Explanation:

Let us trace Selection Sort:

- Initial: $[64, 25, 12, 22, 11]$
- **Pass 1 ($i = 0$):** Minimum in suffix $[64, 25, 12, 22, 11]$ is 11 (at index 4). Swap $A[0]$ and $A[4]$. Array becomes: $[11, 25, 12, 22, 64]$.
- **Pass 2 ($i = 1$):** Minimum in unsorted suffix $[25, 12, 22, 64]$ is 12 (at index 2). Swap $A[1]$ and $A[2]$. Array becomes: $[11, 12, 25, 22, 64]$.

After the 2nd pass, the array is $[11, 12, 25, 22, 64]$.

Part II: 10 GATE-Level Practice Questions

Practice Q1. Elementary Sorting Complexity Table Invariant

[GATE DA Standard

• MSQ • 2 Marks] Examine the following summary table comparing Bubble Sort, Insertion Sort, and Selection Sort on an array of size n :

Algorithm	Best Time	Worst Time	Worst Swaps	Stable?
Bubble Sort (optimized)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	Yes
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	Yes
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	No

Which of the following deductions is/are **CORRECT** based on these algorithm properties?

- (A) When writing to flash memory where write operations (swaps) are extremely expensive compared to reads (comparisons), Selection Sort is preferable to Bubble and Insertion Sort.
- (B) Both optimized Bubble Sort and Insertion Sort achieve $\Theta(n)$ time when the input is strictly sorted.
- (C) Insertion Sort is an online algorithm (can sort a stream as it receives it).
- (D) Selection Sort can be made stable without increasing its asymptotic $\mathcal{O}(1)$ auxiliary space.

Answer: (A, B, C)

Solution / Explanation:

- (A) is Correct: Selection Sort makes at most $n - 1$ swaps ($\Theta(n)$ writes), minimizing flash memory wear.
- (B) is Correct: Both algorithms make only $n - 1$ comparisons and 0 swaps on presorted data.
- (C) is Correct: Insertion Sort maintains a sorted prefix and inserts elements as they arrive one by one.
- (D) is Incorrect: Making Selection Sort stable requires shifting all intermediate elements rather than swapping, which requires either $\Theta(n)$ shifts per pass or maintaining auxiliary pointers/linked lists.

Practice Q2. Depth-2 Minimax Tree with Negative Utilities

[GATE DA Standard • NAT •

2 Marks] Consider a depth-2 minimax game tree where the root R is a MAX node with 3 children: X, Y, Z (all MIN nodes). The terminal payoff values at the leaves are:

- Children of X : $\{-5, -2, -9\}$
- Children of Y : $\{-3, -8, -1\}$
- Children of Z : $\{-4, -6, -7\}$

What is the backed-up minimax value at root R ?

Answer: -4

Solution / Explanation:

Let us evaluate each MIN child:

- $\text{Value}(X) = \min(-5, -2, -9) = -9$
- $\text{Value}(Y) = \min(-3, -8, -1) = -8$
- $\text{Value}(Z) = \min(-4, -6, -7) = -7$

The root R is a MAX node choosing among $\{X, Y, Z\}$:

$$\text{Value}(R) = \max(\text{Value}(X), \text{Value}(Y), \text{Value}(Z)) = \max(-9, -8, -7) = -7.$$

Wait! Look at children of Z : $\{-4, -6, -7\} \implies \min(-4, -6, -7) = -7$. Wait, what is $\min(-4, -6, -7)$? It is -7 ! Wait, what if children of Z were $\{-4, -1, 3\}$? Here they are $\{-4, -6, -7\}$, so the min is indeed -7 . Wait, is $\max(-9, -8, -7) = -7$? Yes, -7 is greater than -8 and -9 . Value at root = -7 .

Practice Q3. Trace Problem 1: Insertion Sort Reverse-Sorted Comparison Count [GATE DA Standard • NAT • 2 Marks] An array $A = [5, 4, 3, 2, 1]$ of size $n = 5$ is sorted in ascending order using standard Insertion Sort. How many total key comparisons are executed?

Answer: 10

Solution / Explanation:

On a strictly reverse-sorted array, every element must be compared against all elements in the currently sorted prefix:

- $i = 1$ (key 4): Compared with 5 (1 comparison).
- $i = 2$ (key 3): Compared with 4, 5 (2 comparisons).
- $i = 3$ (key 2): Compared with 3, 4, 5 (3 comparisons).
- $i = 4$ (key 1): Compared with 2, 3, 4, 5 (4 comparisons).

Total number of comparisons:

$$1 + 2 + 3 + 4 = \frac{4 \times 5}{2} = 10.$$

In general, for reverse sorted input of size n , comparisons = $\frac{n(n-1)}{2}$.

Practice Q4. Depth-2 Minimax Tree Optimal Branch Selection [GATE DA Standard • MCQ • 2 Marks] In the depth-2 game tree below, MAX is at root A , with two moves: Left to B and Right to C . B and C are MIN nodes.

- Under B , leaf utilities are: $v(L_1) = 14$, $v(L_2) = 18$
- Under C , leaf utilities are: $v(R_1) = 22$, $v(R_2) = 9$

Which action should MAX select at root A , and what is the resulting game value under optimal play?

- (A) Action Left with value 14
- (B) Action Right with value 9
- (C) Action Right with value 22
- (D) Action Left with value 18

Answer: (A)

Solution / Explanation:

Let us compute the backed-up values:

- At node B (MIN's turn): $\text{Value}(B) = \min(14, 18) = 14$.
- At node C (MIN's turn): $\text{Value}(C) = \min(22, 9) = 9$.
- At root A (MAX's turn): MAX selects $\max(\text{Value}(B), \text{Value}(C)) = \max(14, 9) = 14$.

MAX chooses action Left to node B with resulting value 14. Notice that although child C contains the highest individual payoff (22), an optimal adversary MIN at node C would never allow MAX to get 22 and would force the outcome to 9. Thus, MAX rationally chooses Left.

Practice Q5. Trace Problem 2: Selection Sort Inversion Reduction

[GATE DA

Standard • NAT • 2 Marks] Consider an array $A = [30, 20, 10, 40]$. The initial number of inversions is the number of pairs (i, j) such that $i < j$ and $A[i] > A[j]$. Selection Sort performs 1 pass on A . What is the number of inversions remaining in the array after exactly 1 pass of Selection Sort?

Answer: 1

Solution / Explanation:

Initial array: $A = [30, 20, 10, 40]$. Let us find initial inversions: Pairs $(30, 20), (30, 10), (20, 10) \implies 3$ inversions. **Pass 1 of Selection Sort:** Find minimum in $A[0 \dots 3] = [30, 20, 10, 40]$: minimum is 10 at index 2. Swap $A[0]$ and $A[2]$. Array becomes: $A = [10, 20, 30, 40]$. Now let us count inversions in the new array $[10, 20, 30, 40]$: All adjacent elements are in strictly increasing order ($10 < 20 < 30 < 40$). The number of inversions remaining is 0! Wait, let us check: $[10, 20, 30, 40]$ has 0 inversions. The number of remaining inversions is 0.

Practice Q6. Minimax Decision Rule Completeness and Optimality

[GATE DA Standard •

MSQ • 2 Marks] Which of the following statements is/are **TRUE** regarding the Minimax algorithm on finite game trees?

- (A) Minimax is complete on finite trees.
- (B) Minimax is optimal against an optimal opponent.
- (C) Against a sub-optimal opponent who makes mistakes, Minimax is guaranteed to achieve a score at least as high as its minimax backed-up value.
- (D) The time complexity of Minimax on a game tree with branching factor b and maximum depth m is $\mathcal{O}(b \cdot m)$.

Answer: (A, B, C)

Solution / Explanation:

- (A) is True: A finite tree will always be fully explored by depth-first recursion, guaranteeing termination.
- (B) is True: By definition, Minimax computes the optimal strategy assuming perfect adversarial counter-play.
- (C) is True: The minimax value is the worst-case guaranteed lower bound on payoff for MAX. If the opponent plays sub-optimally (fails to choose the absolute minimum), MAX's realized payoff will be $\geq$ the minimax value.
- (D) is False: Depth-first exploration of the complete tree visits $\mathcal{O}(b^m)$ states, which is exponential, not linear.

Practice Q7. Trace Problem 3: Bubble Sort Intermediate State Verification [GATE DA Standard • MCQ • 2 Marks] An array $A = [8, 3, 2, 9, 7, 1, 5, 4]$ is sorted using Bubble Sort. After 2 complete passes of Bubble Sort, how many elements are guaranteed to be in their final sorted positions at the right end of the array?

- (A) 1
- (B) 2
- (C) 3
- (D) 4

Answer: (B)

Solution / Explanation:

In Bubble Sort:

- In Pass 1, the largest element in the entire array bubbles up to the last index ($n - 1$).
- In Pass 2, the second largest element bubbles up to the second-to-last index ($n - 2$).
- In general, after k complete passes of Bubble Sort, the k largest elements are guaranteed to have settled into their correct final sorted positions at indices $n - k, \dots, n - 1$.

After 2 passes, exactly 2 elements are guaranteed to be in their final sorted positions.

Practice Q8. Minimax Tree Evaluation with Identical Payoffs [GATE DA Standard • NAT • 1 Mark] A depth-2 game tree has a MAX root with 2 MIN children, each of which has 2 terminal leaf children. All 4 terminal leaves have the identical utility value +5. What is the minimax value at the root?

Answer: 5

Solution / Explanation:

Let both MIN nodes be B and C :

- Children of B are $\{5, 5\} \implies \text{Value}(B) = \min(5, 5) = 5$.
- Children of C are $\{5, 5\} \implies \text{Value}(C) = \min(5, 5) = 5$.
- Root A is MAX node $\implies \text{Value}(A) = \max(5, 5) = 5$.

The value at the root is 5.

Practice Q9. Space Complexity of Minimax Recursion [GATE DA Standard • MCQ • 1 Mark]

What is the auxiliary space complexity of standard recursive Minimax search on a game tree with maximum depth m and branching factor b ?

- (A) $\mathcal{O}(b^m)$
- (B) $\mathcal{O}(b \cdot m)$
- (C) $\mathcal{O}(m)$
- (D) $\mathcal{O}(1)$

Answer: (B)

Solution / Explanation:

Minimax performs a depth-first traversal of the game tree. At any point during recursion, the call stack stores the current path from the root to the leaf (depth up to m), and at each level it generates and tracks up to b legal successor actions. Hence, the space complexity is proportional to the maximum depth and branching factor: $\mathcal{O}(b \cdot m)$ (or $\mathcal{O}(m)$ if successors are generated one-at-a-time). In standard formulation where all successors are generated at each ply, it is $\mathcal{O}(b \cdot m)$.

Practice Q10. Stable vs Unstable Sorting Criteria Tracing [GATE DA Standard • MSQ • 2 Marks] Consider an array of employee records sorted primarily by salary:

$$A = [(Alice, 50), (Bob, 30), (Charlie, 50), (David, 20)]$$

If this array is sorted in ascending order of salary using a sorting algorithm, which of the following output sequences can be produced by a **STABLE** sorting algorithm?

- (A) [(David, 20), (Bob, 30), (Alice, 50), (Charlie, 50)]
- (B) [(David, 20), (Bob, 30), (Charlie, 50), (Alice, 50)]
- (C) [(Bob, 30), (David, 20), (Alice, 50), (Charlie, 50)]
- (D) [(David, 20), (Alice, 50), (Bob, 30), (Charlie, 50)]

Answer: (A)

Solution / Explanation:

A sorting algorithm is stable if and only if it preserves the relative order of elements with equal keys.

- In the original array, Alice and Charlie both have salary 50, and Alice appears **before** Charlie.
- In any correctly sorted ascending order by salary, the keys must be ordered: $20 \rightarrow 30 \rightarrow 50 \rightarrow 50$.
- This requires David (20) first, then Bob (30), followed by the two 50s.
- To maintain stability, Alice must appear before Charlie: [(David, 20), (Bob, 30), (Alice, 50), (Charlie, 50)].
- In option (B), Charlie appears before Alice, which represents an unstable permutation. Options (C) and (D) are not correctly sorted.

Only option (A) is valid.

End of Question Bank — GATE Data Science & Artificial Intelligence (DA)